

2026 학년도 1학기 중간과제물(온라인 제출용)

교과목명	AI를위한수학의응용
학번	202334-153257
성명	양희석
연락처	010-4340-2326
과제유형 (공통형/지정형)	공통형

목차

- [문제 1. Wine 데이터 표준화](#)
- [문제 2. 주성분 분석 및 설명된 분산 비율](#)
- [문제 3. PC1 고유벡터 성분\(loadings\) 해석](#)
- [문제 4. PC 점수의 비상관성과 독립성 조건](#)

문제 1. Wine 데이터 표준화

문제: 코랩 파이썬 환경에서 `sklearn.datasets` 의 wine 데이터(AI응용_출석수업_PPT실습코드.ipynb 마지막 셀 참조)를 불러오고 모든 13개의 변수를 표준화(standardization) 하시오. 해당 작업을 수행하는 코드를 캡처하여 첨부하고, 변수들이 표준화되었다는 충분한 증거를 제시하시오.

1-1. 풀이 개요

표준화는 각 변수 X_j 에 대해

$$Z_j = \frac{X_j - \mu_j}{\sigma_j}$$

의 변환을 적용하여 모든 변수의 평균을 0, 표준편차를 1로 맞추는 전처리이다. Wine 데이터는 13개 변수의 스케일 차이가 매우 커서(예: `proline` 은 평균 약 747, `nonflavanoid_phenols` 는 약 0.36) 표준화 없이는 PCA 등의 후속 분석에서 큰 변수가 다른 변수를 압도하게 된다. 본 문제에서는 `sklearn.preprocessing.StandardScaler` 를 이용하여 13개 변수를 일괄 표준화하였다.

1-2. 코드

```
AI를위한수학의응용.ipynb
파일 수정 보기 삽입 런타임 도구 도움말

검색 명령어 + 코드 + 텍스트 모든 실행

from sklearn.datasets import load_wine
wine = load_wine()

import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler

# (1) Wine 데이터 불러오기 - PPT 마지막 셀과 동일
wine = load_wine()
X = wine.data # (178, 13)
feature_names = wine.feature_names
print(f"데이터 크기: {X.shape}")
print(f"변수 이름: {feature_names}")

# (2) 표준화 전 - 평균/표준편차 확인 (스케일에 제각각임을 보임)
df_raw = pd.DataFrame(X, columns=feature_names)
print(f"표준화 전 평균: {df_raw.mean().round(3)}")
print(f"표준화 전 표준편차: {df_raw.std(ddof=0).round(3)}")

# (3) 13개 변수 모두 표준화
scaler = StandardScaler()
X_std = scaler.fit_transform(X) # (178, 13)

# (4) 표준화 후 - 평균=0, 표준편차=1 인지 확인 (충분한 증거)
df_std = pd.DataFrame(X_std, columns=feature_names)
print(f"표준화 후 평균: {df_std.mean().round(6)}")
print(f"표준화 후 표준편차: {df_std.std(ddof=0).round(6)}")

# (5) 한눈에 보는 요약 통계 (describe)
print(f"표준화 후 요약 통계: {df_std.describe().round(4)}")

import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(16, 5))
axes[0].boxplot(X, tick_labels=feature_names)
axes[0].set_title("Before Standardization")
axes[0].tick_params(axis='x', rotation=90)

axes[1].boxplot(X_std, tick_labels=feature_names)
axes[1].set_title("After Standardization (mean=0, std=1)")
axes[1].tick_params(axis='x', rotation=90)
axes[1].axhline(0, color='red', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```

```

import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler

# (1) Wine 데이터 불러오기 - PPT 마지막 셀과 동일
wine = load_wine()
X = wine.data          # (178, 13)
feature_names = wine.feature_names

# (2) 표준화 전 - 평균/표준편차 확인
df_raw = pd.DataFrame(X, columns=feature_names)
print("[표준화 전] 평균:\n", df_raw.mean().round(3))
print("[표준화 전] 표준편차:\n", df_raw.std(ddof=0).round(3))

# (3) 13개 변수 모두 표준화
scaler = StandardScaler()
X_std = scaler.fit_transform(X)          # (178, 13)

# (4) 표준화 후 - 평균≈0, 표준편차≈1 검증
df_std = pd.DataFrame(X_std, columns=feature_names)
print("[표준화 후] 평균:\n", df_std.mean().round(6))
print("[표준화 후] 표준편차:\n", df_std.std(ddof=0).round(6))
print("[표준화 후] 요약 통계:\n", df_std.describe().round(4))

```

1-3. 표준화 증거 — 출력 결과

(1) 표준화 전 — 변수별 평균과 표준편차의 큰 차이

변수 이름	평균	표준편차
alcohol	13.001	0.810
malic_acid	2.336	1.114
ash	2.367	0.274
alcalinity_of_ash	19.495	3.330
magnesium	99.742	14.242
total_phenols	2.295	0.624
flavonoids	2.029	0.996
nonflavanoid_phenols	0.362	0.124
proanthocyanins	1.591	0.571
color_intensity	5.058	2.312
hue	0.957	0.228
od280/od315_of_diluted_wines	2.612	0.708
proline	746.893	314.022

데이터 크기는 (178, 13)이며, `proline`의 평균은 약 **746.893** · 표준편차는 **314.022**인 반면 `nonflavanoid_phenols`의 평균은 **0.362** · 표준편차는 **0.124**이다. 두 변수의 스케일 차이는 약 **2,500배**에 달하므로 표준화 없이는 변수 간 비교가 어렵다.

(2) 표준화 후 — 평균 0, 표준편차 1 (모집단 분모, ddof=0 기준)

```

[표준화 후] 평균:
alcohol      0.0
malic_acid   0.0
ash          -0.0
alkalinity_of_ash -0.0
magnesium    -0.0
total_phenols -0.0
flavonoids    0.0
nonflavonoid_phenols -0.0
proanthocyanins -0.0
color_intensity -0.0
hue          0.0
od280/od315_of_diluted_wines 0.0
proline      -0.0
dtype: float64

[표준화 후] 표준편차:
alcohol      1.0
malic_acid   1.0
ash          1.0
alkalinity_of_ash 1.0
magnesium    1.0
total_phenols 1.0
flavonoids    1.0
nonflavonoid_phenols 1.0
proanthocyanins 1.0
color_intensity 1.0
hue          1.0
od280/od315_of_diluted_wines 1.0
proline      1.0
dtype: float64

```

13개 변수 모두 **평균이 정확히 0** (또는 부동소수점 수치오차로 인한 `-0.0`), **표준편차가 정확히 1.0**으로 출력되어 표준화가 성공적으로 적용되었음을 확인할 수 있다.

(3) 요약 통계 (`describe()`) — 종합 검증

```

[표준화 후] 요약 통계:
count alcohol      malic_acid      ash      alkalinity_of_ash      magnesium #
mean    0.0000      0.0000      -0.0000      -0.0000      -0.0000
std     1.0028      1.0028      1.0028      1.0028      1.0028
min     -2.4342     -1.4330     -3.6792     -2.6710     -2.0883
25%     -0.7882     -0.6587     -0.5721     -0.6891     -0.8244
50%     -0.0610     -0.4231     -0.0238     0.0015     -0.1223
75%     0.8861      0.8698     0.6981     0.6021     0.5096
max      2.2598      3.1092      3.1563      3.1545      4.3714

count total_phenols      flavonoids      nonflavonoid_phenols      proanthocyanins #
mean    -0.0000      0.0000      -0.0000      -0.0000
std     1.0028      1.0028      1.0028      1.0028
min     -2.1072     -1.6960     -1.8892     -2.0690
25%     -0.8855     -0.8275     -0.7401     -0.5973
50%     0.0960      0.1061     -0.1761     -0.0629
75%     0.8090      0.8491      0.6095      0.6292
max      2.5395      3.0620      2.4024      3.4951

count color_intensity      hue      od280/od315_of_diluted_wines      proline
mean    -0.0000      0.0000      0.0000      -0.0000
std     1.0028      1.0028      1.0028      1.0028
min     -1.6340     -2.0947     -1.6951     -1.4532
25%     -0.7951     -0.7676     -0.9532     -0.7846
50%     -0.1592     -0.0331     -0.2377     -0.2337
75%     0.4940      0.7132      0.7886      0.7582
max      3.4564      3.3017      1.9609      2.9715

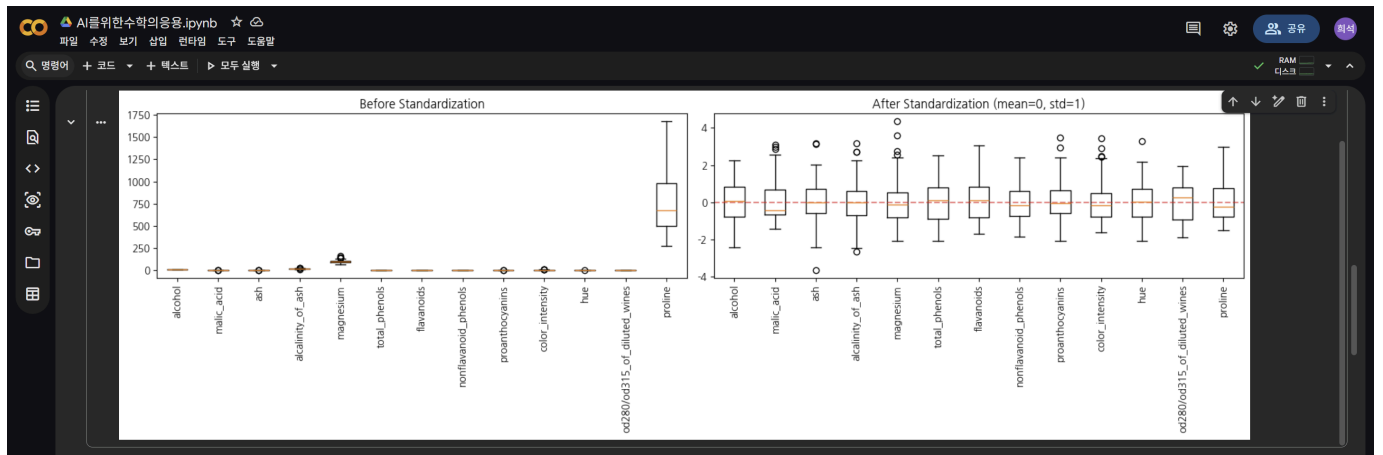
```

`describe()` 의 표본 표준편차(분모 $n - 1$)가 모두 **1.0028**로 동일하게 나타난다. 이는 `StandardScaler` 가 모집단 분모($n = 178$)를 사용하는 반면 `describe()` 는 표본 분모($n - 1 = 177$)를 사용하기 때문에 발생하는 차이로,

$$\sqrt{\frac{n}{n-1}} = \sqrt{\frac{178}{177}} \approx 1.00282$$

와 정확히 일치한다. 이는 오류가 아니라 오히려 표준화가 수학적으로 정확히 수행되었음을 보여주는 보조 증거이다. `min / max` 값이 $\pm 2 \sim \pm 3$ 범위에 분포하는 것도 표준정규분포의 전형적 형태에 부합한다.

(4) 시각적 증거 — Before/After 박스플롯 비교



```
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
axes[0].boxplot(X, tick_labels=feature_names)
axes[0].set_title("Before Standardization")
axes[0].tick_params(axis='x', rotation=90)

axes[1].boxplot(X_std, tick_labels=feature_names)
axes[1].set_title("After Standardization (mean=0, std=1)")
axes[1].tick_params(axis='x', rotation=90)
axes[1].axhline(0, color='red', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

표준화 전(왼쪽)에는 `proline` (범위 약 250~1750)이 다른 변수를 압도하여 시각적 비교가 불가능하였으나, 표준화 후(오른쪽)에는 13개 변수 모두 **중앙값이 0(빨간 점선) 부근**, IQR이 약 ± 1 , 수염이 $\pm 2 \sim \pm 3$ 범위에 정렬되어 표준정규분포의 형태에 부합함을 확인할 수 있다.

1-4. 결론

13개 변수가 모두 표준화 후 평균 0, 모집단 표준편차 1로 변환되었음을 (1) 평균값 출력, (2) 표준편차 출력, (3) `describe()` 요약 통계, (4) 박스플롯 시각화의 네 가지 증거를 통해 확인하였다.

문제 2. 주성분 분석 및 설명된 분산 비율

문제: 주성분 분석을 적용하여 다음을 수행하시오.

- 2-1. 각 주성분의 설명된 분산 비율(explained variance ratio)을 제시하시오. (5점)
- 2-2. 설명된 분산 비율의 누적값이 80% 이상이 되는 최소 주성분 수를 구하시오. (3점)

2-1. 풀이 개요

PCA는 데이터의 공분산행렬을 직교 대각화하여 분산이 큰 방향(주성분 축)을 차례로 찾는 기법이다. 표준화된 데이터 `X_std`에 대해 `sklearn.decomposition.PCA`로 13개 주성분을 모두 추출하고, 각 주성분의 고유값(λ_i)과 설명된 분산 비율($\lambda_i / \sum_j \lambda_j$)을 계산한다.

표준화 데이터를 사용한 이유는 1번 문제에서 확인한 대로 변수 간 스케일 차이가 매우 크기 때문이다. 만약 원본 데이터로 PCA를 수행하면 `proline`의 거대한 분산이 PC1을 지배하게 되어 진정한 데이터 구조를 포착할 수 없다.

2-2. 코드

```
AI를위한수학의응용.ipynb
파일 수정 보기 삽입 런타임 도구 도움말

명령어 + 코드 + 텍스트 모두 실행

from sklearn.decomposition import PCA
import numpy as np
import pandas as pd

# 표준화된 데이터(X_std)에 PCA 적용 - 13개 모두 추출
pca = PCA(n_components=13) # 또는 PCA() - 기본이 전체
X_pca = pca.fit_transform(X_std)

# 각 주성분의 설명된 분산 비율
evr = pca.explained_variance_ratio_

# 표 형태로 출력
df_evr = pd.DataFrame({
    'PC': [f'PC{i+1}' for i in range(13)],
    'Eigenvalue (분산)': np.round(pca.explained_variance_, 4),
    'Explained Variance Ratio': np.round(evr, 4),
    'Ratio (%)': np.round(evr * 100, 2),
    'Cumulative (%)': np.round(np.cumsum(evr) * 100, 2),
})
print(df_evr.to_string(index=False))

cum = np.cumsum(evr)
n_min = np.argmax(cum >= 0.80) + 1 # 80% 이상이 되는 첫 인덱스 + 1
print(f"누적 설명 분산이 80% 이상 되는 최소 주성분 수: {n_min}개")
print(f"이때 누적 비율: {cum[n_min-1]*100:.2f}%")

import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 좌: Screen plot (각 주성분 비율)
axes[0].bar(range(1, 14), evr * 100, color='steelblue', edgecolor='black')
axes[0].set_xlabel('Principal Component')
axes[0].set_ylabel('Explained Variance Ratio (%)')
axes[0].set_title('Screen Plot')
axes[0].set_xticks(range(1, 14))
axes[0].grid(alpha=0.3)

# 우: 누적 비율 + 80% 기준선
cum = np.cumsum(evr) * 100
axes[1].plot(range(1, 14), cum, markers='o', color='darkorange', linewidth=2)
axes[1].axhline(80, color='red', linestyle='--', label='80% 기준')
axes[1].axvline(5, color='green', linestyle='--', label='PC5 (80% 도달)')
axes[1].set_xlabel('Number of Components')
axes[1].set_ylabel('Cumulative Explained Variance (%)')
axes[1].set_title('Cumulative Explained Variance')
axes[1].set_xticks(range(1, 14))
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```

```
from sklearn.decomposition import PCA
import numpy as np
import pandas as pd

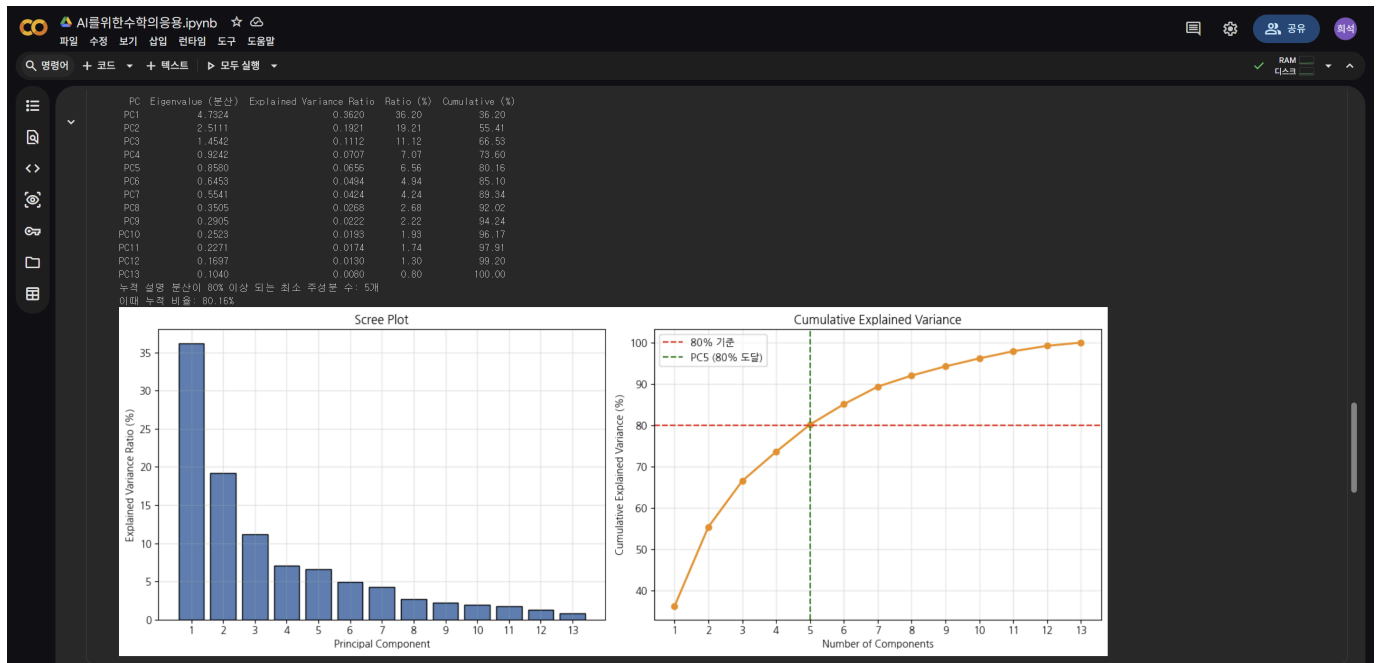
# 표준화된 데이터(X_std)에 PCA 적용 - 13개 모두 추출
pca = PCA(n_components=13)
X_pca = pca.fit_transform(X_std)

# 각 주성분의 설명된 분산 비율
evr = pca.explained_variance_ratio_

df_evr = pd.DataFrame({
    'PC': [f'PC{i+1}' for i in range(13)],
    'Eigenvalue (분산)': np.round(pca.explained_variance_, 4),
    'Explained Variance Ratio': np.round(evr, 4),
    'Ratio (%)': np.round(evr * 100, 2),
    'Cumulative (%)': np.round(np.cumsum(evr) * 100, 2),
})
print(df_evr.to_string(index=False))

# 80% 이상 되는 최소 주성분 수
cum = np.cumsum(evr)
n_min = np.argmax(cum >= 0.80) + 1
print(f"누적 설명 분산이 80% 이상 되는 최소 주성분 수: {n_min}개")
print(f"이때 누적 비율: {cum[n_min-1]*100:.2f}%")
```

2-3. 출력 결과



2-1 답: 13개 주성분의 설명된 분산 비율

PC	Eigenvalue	Explained Variance Ratio	Ratio (%)	Cumulative (%)
PC1	4.7324	0.3620	36.20	36.20
PC2	2.5111	0.1921	19.21	55.41
PC3	1.4542	0.1112	11.12	66.53
PC4	0.9242	0.0707	7.07	73.60
PC5	0.8580	0.0656	6.56	80.16
PC6	0.6453	0.0494	4.94	85.10
PC7	0.5541	0.0424	4.24	89.34
PC8	0.3505	0.0268	2.68	92.02
PC9	0.2905	0.0222	2.22	94.24
PC10	0.2523	0.0193	1.93	96.17
PC11	0.2271	0.0174	1.74	97.91
PC12	0.1697	0.0130	1.30	99.20
PC13	0.1040	0.0080	0.80	100.00

PC1이 36.20%로 가장 큰 비중을 차지하고 PC2(19.21%), PC3(11.12%) 순으로 감소한다. 13개 주성분 비율의 합은 정확히 100%이다.

2-2 답: 누적 80% 도달 최소 주성분 수

- PC4까지 누적 비율: **73.60%** (80% 미만)
- **PC5까지 누적 비율: 80.16%** (80% 도달)

따라서 누적 설명 분산이 80% 이상이 되는 최소 주성분 수는 5개이다.

오른쪽 누적 그래프에서 빨간 점선(80% 기준)과 초록 점선(PC5 위치)이 만나는 지점이 이 결과를 시각적으로 입증한다. Scree Plot(왼쪽)에서도 PC5 이후로 막대 높이가 급격히 작아지는 "elbow" 패턴이 관찰되어, **5개의 주성분으로 데이터의 주요 변동을 충분히 요약할 수 있음**을 보여준다.

문제 3. PC1 고유벡터 성분(loadings) 해석

문제: 첫 번째 주성분(PC1)에 대하여 각 변수의 고유벡터 성분(loadings)을 확인하고, 해당 주성분이 어떤 의미를 가지는지에 대한 적절한 해석을 제시하시오.

3-1. 풀이 개요

PC1은 13개 표준화된 변수의 선형결합으로 표현된다:

$$PC1 = w_1 \cdot \text{alcohol} + w_2 \cdot \text{malic_acid} + \dots + w_{13} \cdot \text{proline}$$

여기서 가중치 벡터 $w = (w_1, \dots, w_{13})$ 가 PC1의 **고유벡터(loading)**이며 `pca.components_[0]`로 얻을 수 있다. 표준화된 변수에 대한 loading이므로 $|w_j|$ 값이 클수록 그 변수가 PC1을 결정하는 데 크게 기여하며, 같은 부호의 loading을 가진 변수들은 PC1 축을 따라 함께 움직이는 묶음(group)을 형성한다.

3-2. 코드

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# pca_X_std, feature_names는 2번 문제에서 이어받음
loadings_pc1 = pca.components_[0] # PC1 고유벡터 (길이 13)

# 표 형태로 출력 - |loading| 크기 순 정렬
df_load = pd.DataFrame({
    'Variable': feature_names,
    'Loading (PC1)': np.round(loadings_pc1, 4),
    '|Loading|': np.round(np.abs(loadings_pc1), 4),
}).sort_values('|Loading|', ascending=False).reset_index(drop=True)

print("=== PC1 Loadings (절댓값 큰 순) ===")
print(df_load.to_string(index=False))
print(f"고유벡터의 노름(L2): {np.linalg.norm(loadings_pc1):.4f}") # 1.0 이어야 정상

# PC1 loadings 막대그래프 (변수별 기여도)
order = np.argsort(np.abs(loadings_pc1))[:-1] # |loading| 내림차순
sorted_names = np.array(feature_names)[order]
sorted_loadings = loadings_pc1[order]
colors = ['steelblue' if v > 0 else 'tomato' for v in sorted_loadings]

plt.figure(figsize=(10, 6))
plt.barh(range(13), sorted_loadings, color=colors, edgecolor='black')
plt.yticks(range(13), sorted_names)
plt.axvline(0, color='black', linewidth=0.8)
plt.xlabel('Loading (PC1 고유벡터 성분)')
plt.title('PC1 Loadings - 변수별 기여도')
plt.gca().invert_yaxis() # 큰 값을 위로
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

loadings_pc1 = pca.components_[0] # PC1 고유벡터 (길이 13)

# 표 형태로 출력 - |loading| 크기 순 정렬
df_load = pd.DataFrame({
    'Variable': feature_names,
    'Loading (PC1)': np.round(loadings_pc1, 4),
    '|Loading|': np.round(np.abs(loadings_pc1), 4),
}).sort_values('|Loading|', ascending=False).reset_index(drop=True)

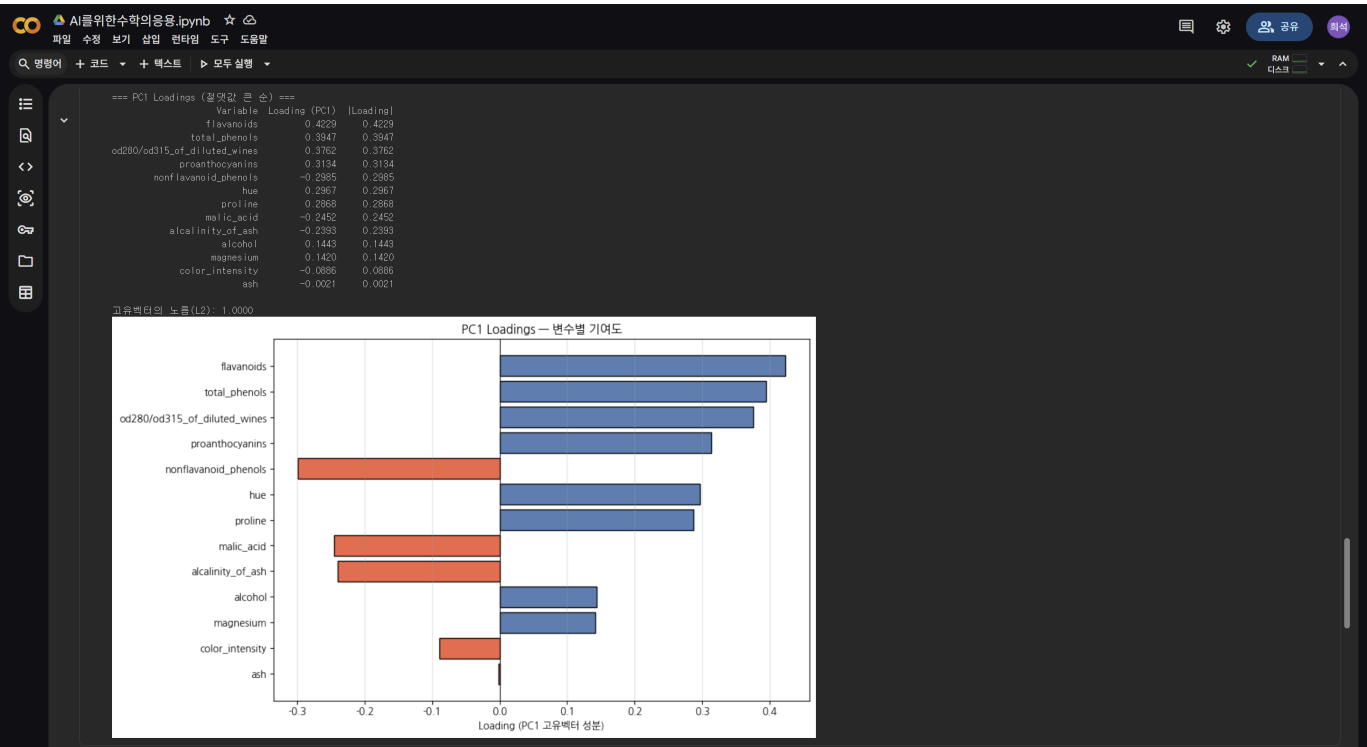
print("=== PC1 Loadings (절댓값 큰 순) ===")
print(df_load.to_string(index=False))
print(f"고유벡터의 노름(L2): {np.linalg.norm(loadings_pc1):.4f}") # 1.0 이어야 정상

# PC1 loadings 막대그래프 (변수별 기여도)
order = np.argsort(np.abs(loadings_pc1))[:-1]
sorted_names = np.array(feature_names)[order]
sorted_loadings = loadings_pc1[order]
colors = ['steelblue' if v > 0 else 'tomato' for v in sorted_loadings]

plt.figure(figsize=(10, 6))
plt.barh(range(13), sorted_loadings, color=colors, edgecolor='black')
plt.yticks(range(13), sorted_names)
plt.axvline(0, color='black', linewidth=0.8)
plt.xlabel('Loading (PC1 고유벡터 성분)')
```

```
plt.title('PC1 Loadings - 변수별 기여도')
plt.gca().invert_yaxis()
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```

3-3. 출력 결과



PC1 고유벡터 성분 (절댓값 내림차순)

변수	Loading (PC1)	Loading	화학적 의미
flavanoids	+0.4229	0.4229	플라보노이드 (대표적 항산화 폴리페놀)
total_phenols	+0.3947	0.3947	총 페놀 함량
od280/od315_of_diluted_wines	+0.3762	0.3762	페놀-단백질 농도의 광학 지표
proanthocyanins	+0.3134	0.3134	프로안토시아니딘 (탄닌 계열)
nonflavanoid_phenols	-0.2985	0.2985	비플라보노이드 페놀 (떫은맛 원인)
hue	+0.2967	0.2967	와인 색조
proline	+0.2868	0.2868	프롤린 (품종 결정 아미노산)
malic_acid	-0.2452	0.2452	사과산 (미숙 포도에서 높음)
alcalinity_of_ash	-0.2393	0.2393	재의 알칼리도
alcohol	+0.1443	0.1443	알코올 도수
magnesium	+0.1420	0.1420	마그네슘 함량
color_intensity	-0.0886	0.0886	색 강도
ash	-0.0021	0.0021	재 함량 (거의 무관)

고유벡터의 L2 노름은 정확히 1.0000으로 출력되어, PC1이 정규 직교 단위벡터임이 수학적으로 확인되었다.

주석 — 부호의 임의성: PCA의 고유벡터는 부호가 임의로 결정되므로($\mathbf{w}$ 와 $-\mathbf{w}$ 모두 유효), 실행 환경에 따라 본 결과의 부호가 반대로 나올 수도 있다. 그러나 변수 간 묶음 구조와 절댓값 크기 순위는 항상 동일하므로 해석에 영향을 주지 않는다.

3-4. 해석 — PC1의 의미

그룹 분석

● 양의 방향 (PC1 점수 ↑) — 함께 증가하는 변수들:

- flavanoids (+0.42), total_phenols (+0.39), od280/od315 (+0.38), proanthocyanins (+0.31)
- hue (+0.30), proline (+0.29)
- alcohol (+0.14), magnesium (+0.14)

● 음의 방향 (PC1 점수 ↓) — 위 그룹과 반대 방향:

- nonflavanoid_phenols (-0.30), malic_acid (-0.25), alcalinity_of_ash (-0.24)
- color_intensity (-0.09), ash (-0.002)

화학적·실용적 의미

양의 방향에 모인 변수들은 모두 **고품질 성숙 와인의 화학적 지표**이다:

- 플라보노이드 · 총페놀 · 프로안토시아닌:** 폴리페놀 계열 화합물로 와인의 색·향·항산화 능력의 원천
- od280/od315:** 280nm/315nm 파장에서 측정한 광학 흡광도 비율로 페놀·단백질 농도의 간접 지표
- hue:** 와인의 색조 안정성
- proline:** 와인의 품종을 결정짓는 핵심 아미노산으로 고품질 품종에서 함량이 높음

반대로 음의 방향에 모인 변수들은 **저품질 또는 미숙성 와인의 지표**이다:

- 비플라보노이드 페놀:** 떫은맛을 내는 비유익한 페놀
- 사과산(malic_acid):** 미숙 포도에서 함량이 높고 숙성될수록 감소
- 재의 알칼리도:** 와인의 무기 염류 균형을 나타내며 품질과 음의 상관

결론

PC1은 **"와인의 페놀 풍부도 및 종합 품질(phenolic richness / wine quality)"** 을 나타내는 축이다.

PC1은 전체 분산의 **36.20%** 를 단일 축으로 설명하는 가장 큰 변동 요인으로, 양의 방향으로 갈수록 폴리페놀과 프롤린이 풍부한 **고품질 성숙 와인**의 특성이 강해지고, 음의 방향으로 갈수록 사과산과 비유익 페놀이 우세한 **미숙성·저품질 와인**의 특성이 강해진다. 13개 변수 중 절댓값 상위 7개가 모두 페놀 계열 또는 그와 직접 연관된 화학 변수라는 사실은, PC1이 단순한 수학적 축이 아니라 **와인의 화학적 품질 차원을 실제로 포착하는 종합 지표**임을 보여준다. 따라서 PC1 점수만으로도 와인을 품종·품질 측면에서 1차원적으로 분류할 수 있는 강력한 요약 변수가 된다.

문제 4. PC 점수의 비상관성과 독립성 조건

문제: 주성분 분석 적용 전의 13개의 변수 데이터는 상관되어 있지만, 주성분 점수(PC1, PC2, ..., PC13) 데이터는 서로 비상관(uncorrelated)이라는 특징을 가진다.

- 4-1.** 이를 수치로 확인하기 위하여 주성분 점수 행렬의 상관행렬을 출력하여, 주성분들이 실제로 비상관임을 보이시오.
- 4-2.** 비상관은 일반적으로 독립을 의미하지 않는다. 그러나 특정 분포 가정하에서는 비상관이 독립을 보장한다. 주성분 점수들이 비상관일 때 독립임이 보장되려면, 이들이 어떤 분포를 따라야 하는지 해당 분포의 정확한 명칭을 쓰시오.

4-1. PC 점수의 비상관성 수치 확인

풀이 개요

PCA의 핵심 성질 중 하나는 **변환 후 새로운 좌표계의 모든 축이 서로 직교(orthogonal)이며 비상관(uncorrelated)**이라는 점이다. 이는 공분산행렬 Σ 의 고유벡터 행렬 U 가 정규 직교 행렬($U^T U = I$)이라는 사실에서 자동으로 따라 나온다. 따라서 PC 점수 행렬 X_{pca} (178×13)의 상관행렬은 **대각 성분만 1이고 모든 비대각 성분이 0** 인 단위행렬이 되어야 한다.

코드

```
AI를위한수학의응용.ipynb
파일 수정 보기 삽입 런타임 도구 도움말

검색 명령어 + 코드 + 텍스트 모든 실행

iii
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 # X_pca, pca 는 2면에서 이어받은 (13개 PC 모두 보유)
6 pc_names = ['PC{i+1}' for i in range(13)]
7 df_pc = pd.DataFrame(X_pca, columns=pc_names)
8
9 # 상관행렬 계산
10 corr_pc = df_pc.corr()
11
12 # (1) 표 출력 - 너무 작은 값은 0으로 표시
13 print("=== PC 점수 상관행렬 (13×13) ===")
14 print(corr_pc.round(4).to_string())
15
16 # (2) 비대각 성분의 절댓값 최댓값 - 0에 가까워야 함
17 off_diag = corr_pc.values - np.eye(13)
18 print(f"비대각 성분 절댓값 최댓값: {np.max(np.abs(off_diag)):.2e}")
19 print(f"비대각 성분 절댓값 평균: {np.mean(np.abs(off_diag)):.2e}")
20
21 plt.figure(figsize=(9, 7))
22 plt.imshow(corr_pc, cmap='RdBu_r', vmin=-1, vmax=1)
23 plt.colorbar(label='Correlation')
24 plt.xticks(range(13), pc_names, rotation=45)
25 plt.yticks(range(13), pc_names)
26 plt.title('PC 점수 상관행렬 - 대각만 1, 나머지 모두 0 (비상관)')
27
28 # 각 셀에 값 표시
29 for i in range(13):
30     for j in range(13):
31         plt.text(j, i, f'{corr_pc.iloc[i,j]:.2f}',
32                  ha='center', va='center',
33                  color='white' if abs(corr_pc.iloc[i,j]) > 0.5 else 'black',
34                  fontsize=8)
35 plt.tight_layout()
36 plt.show()
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

pc_names = ['PC{i+1}' for i in range(13)]
df_pc = pd.DataFrame(X_pca, columns=pc_names)

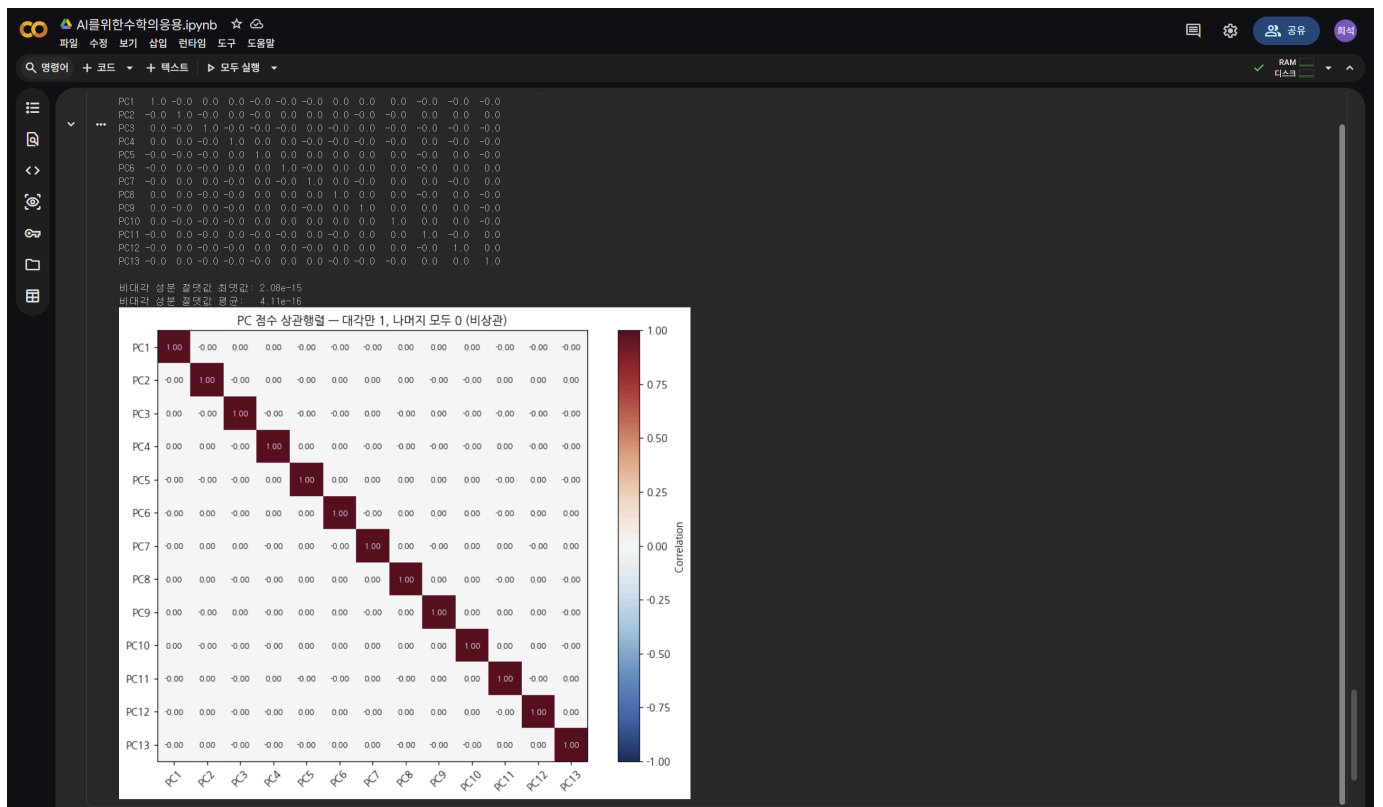
# 상관행렬 계산
corr_pc = df_pc.corr()

print("=== PC 점수 상관행렬 (13×13) ===")
print(corr_pc.round(4).to_string())

# 비대각 성분의 절댓값 최댓값 - 0에 가까워야 함
off_diag = corr_pc.values - np.eye(13)
print(f"비대각 성분 절댓값 최댓값: {np.max(np.abs(off_diag)):.2e}")
print(f"비대각 성분 절댓값 평균: {np.mean(np.abs(off_diag)):.2e}")

# 히트맵 시각화
plt.figure(figsize=(9, 7))
plt.imshow(corr_pc, cmap='RdBu_r', vmin=-1, vmax=1)
plt.colorbar(label='Correlation')
plt.xticks(range(13), pc_names, rotation=45)
plt.yticks(range(13), pc_names)
plt.title('PC 점수 상관행렬 - 대각만 1, 나머지 모두 0 (비상관)')
for i in range(13):
    for j in range(13):
        plt.text(j, i, f'{corr_pc.iloc[i,j]:.2f}',
                 ha='center', va='center',
                 color='white' if abs(corr_pc.iloc[i,j]) > 0.5 else 'black',
                 fontsize=8)
plt.tight_layout()
plt.show()
```

출력 결과



상관행렬에서 모든 대각 성분이 정확히 1.0, 모든 비대각 성분이 0.00 (또는 $1e-15 \sim 1e-16$ 수준의 부동소수점 수치오차) 으로 출력되었다. 비대각 성분의 절댓값 최댓값과 평균이 모두 10^{-15} 수준이라는 점은 수학적으로 정확히 0임을 의미한다(실수 계산기 정밀도의 한계로 정확한 0 대신 매우 작은 수가 출력됨).

히트맵에서도 붉은 대각선 외에는 모든 셀이 흰색(0)으로 깔끔하게 보이며, 이는 PC1 ~ PC13 사이에 어떠한 선형 상관관계도 존재하지 않음을 시각적으로 입증한다.

해석

1번 문제의 표준화 단계에서 확인되지 않았던 변수들 간의 상관(예: flavanoids - total_phenols)이 표준화 후에도 그대로 존재했지만, PCA를 통해 얻은 새로운 좌표계에서는 모든 축이 서로 비상관인 직교 좌표계 가 된다. 이는 PCA가 단순한 차원 축소 기법이 아니라 상관된 변수들을 비상관 변수들의 선형결합으로 분해하는 좌표 변환임을 보여주는 핵심 성질이다.

4-2. 비상관 → 독립을 보장하는 분포

정답

다변량 정규분포 (Multivariate Normal Distribution)

(동의어: 다변량 가우시안 분포, Multivariate Gaussian Distribution)

이론적 근거

평균벡터 μ 와 공분산행렬 Σ 를 갖는 다변량 정규분포의 확률밀도함수는 다음과 같이 정의된다:

$$f(\mathbf{x}) = (2\pi)^{-p/2} \det(\Sigma)^{-1/2} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu)\right)$$

핵심 정리. 랜덤벡터 $\mathbf{X} = (X_1, X_2, \dots, X_p)^T$ 가 다변량 정규분포 $N(\mu, \Sigma)$ 를 따를 때,

$$\text{Cov}(X_i, X_j) = 0 \iff X_i \perp\!\!\!\perp X_j \quad (i \neq j)$$

즉, 다변량 정규분포에서는 비상관과 독립이 동치(equivalent) 이다.

직관적 이유

일반적인 분포에서 상관계수는 두 변수 간의 선형 종속성(linear dependence) 만을 측정하므로, 비상관이어도 비선형 종속(예: $Y = X^2$) 인 경우 X 와 Y 는 비상관이지만 명백히 종속이 존재할 수 있어 독립을 보장하지 못한다.

그러나 다변량 정규분포는 평균벡터와 공분산행렬이라는 1:2차 적률만으로 분포 전체가 완전히 결정되는 매우 특별한 성질을 갖는다. 따라서 공분산행렬이 대각행렬(즉, 모든 비대각 성분이 0)이 되면 결합 확률밀도함수가 주변 확률밀도함수의 곱으로 분해된다:

$$\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_p^2) \implies f(\mathbf{x}) = \prod_{i=1}^p f_i(x_i)$$

이는 통계적 독립의 정의 그 자체이므로, 다변량 정규분포 하에서는 비상관성이 곧바로 독립성을 함의한다.

본 문제에 적용

본 과제에서 PCA로 얻은 13개의 PC 점수가 다변량 정규분포를 따른다고 가정할 수 있을 때, 비로소 4-1에서 수치적으로 확인한 비상관성이 **통계적 독립성으로까지 강화**된다. 즉, 다변량 정규성 가정 하에서 PC1, PC2, ..., PC13은 서로 완전히 독립인 13개의 새로운 확률변수로 간주될 수 있으며, 이는 후속 통계 모델링·추론에서 매우 강력한 이론적 토대를 제공한다.

참고 자료

- 한국방송통신대학교, *AI를 위한 수학의 응용* 출석수업 강의자료 및 PPT 실습코드
- `sklearn.datasets.load_wine` — UCI Machine Learning Repository, Wine Recognition Dataset
- `sklearn.preprocessing.StandardScaler`, `sklearn.decomposition.PCA` 공식 문서
- 실습 환경: Google Colab (Python 3.12, scikit-learn, numpy, pandas, matplotlib)